Injection de contenu arbitraire (CRLF) dans les e-mails de notification des formulaires publics

Programme : CyberAar — AarMe (bug bounty) · **Actif visé** : API formulaires — `cyberaar.io/api/*` (classe **Standard**)
Endpoints : `POST https://cyberaar.io/api/contact` et `POST https://cyberaar.io/api/apply`
Classe de la faille : **CWE-93 — Injection CRLF** · **CVSS v4.0** :
`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/S:X/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N` = **5,3** · **Impact métier** : 4/10 → **score final CyberAar** : √(5,3 × 4) = **4,6 (Moyenne)**
Découverte : 22/10/2026 · **Preuve** : accusé de réception reçu avec le nom tronqué (capture §3)

1. Résumé

Les champs des formulaires publics de CyberAar ne sont pas nettoyés des caractères de fin de ligne (`CR`/`LF`) avant d'être utilisés dans les e-mails générés par la plateforme. La valeur fournie par l'attaquant est donc insérée **brute** dans le message, ce qui permet d'ajouter des lignes arbitraires au contenu de l'e-mail — y compris des en-têtes d'e-mail si le champ concerné atterrit dans un en-tête.

La preuve est directe et reproductible : le message d'accusé de réception renvoyé à l'expéditeur commence par « Bonjour, » **sans le nom** alors que le nom soumis contenait un retour à la ligne : le champ a été tronqué exactement à l'emplacement du `\r\n` injecté. Aucun nettoyage, aucun échappement, aucun refus.

Le risque principal n'est pas la plateforme elle-même mais la **boîte de réception de l'équipe CyberAar** : un attaquant peut introduire dans un message de confiance (envoyé depuis le site officiel) un faux bloc « De : … », une fausse signature, de faux liens, un faux avis de sécurité, ou une demande de désactivation de compte. C'est une voie de **hameçonnage/impersonation à l'encontre du personnel de triage**, dans un canal perçu comme interne. Le même défaut rend l'accusé de réception malformé (contenu tronqué).

2. Actif et périmètre

Actif	URL	Classe
API formulaires	`cyberaar.io/api/*`	Standard (28,3/100)

Les deux endpoints testés sont publics et non authentifiés (aucune session, aucun jeton) : ils n'exigent ni identité validée ni compte (le seul filtre est le rejet des adresses « jetables », qui n'est pas un contrôle de sécurité mais une commodité).

3. Reproduction

3.1 Requête (une seule requête suffit)

```
curl -s -X POST https://cyberaar.io/api/contact \
  -H 'Content-Type: application/json' \
  -H 'Origin: https://cyberaar.io' \
```

```
  -d '{"name":"Test CyberAar\r\nBcc: bbtest4926@maxxspace.com",
       "email":"researcher+test@example-technique.org",
       "subject":"Test programme",
       "message":"Message de test - rapport de bug bounty."}'
```

Réponse : `HTTP 200 {"success":true}` — la charge est acceptée sans le moindre refus (alors que le champ `email`, lui, est validé : un CRLF dans `email` renvoie `400 {"error":"invalid_email"}`).

3.2 Preuve de l'insertion brute

L'accusé de réception reçu par l'expéditeur (capture §5) commence par :

```
Bonjour,

Nous avons bien reçu votre message.

Notre équipe vous répondra dans les 24-48 heures ouvrées.
```

Le nom soumis était `Test CyberAar\r\nBcc: …` : le message affiche « Bonjour, » puis un retour à la ligne, et **le reste de la charge a disparu du corps** — il a été consommé comme un saut de ligne, exactement comme un saut de ligne injections dans un corps d'e-mail. La valeur n'est ni échappée, ni nettoyée, ni refusée : **elle est insérée telle quelle**.

3.3 Vérification côté en-têtes (à confirmer par l'équipe CyberAar)

Deux questions ne peuvent pas être tranchées depuis l'extérieur et sont **explicitement laissées à la vérification interne** (les étapes sont fournies) :

Question	Comment CyberAar peut le vérifier en 2 minutes
La charge atteint-elle un **en-tête** de l'e-mail (et non seulement le corps) ?	Ouvrir l'e-mail reçu par l'équipe (`contact@cyberaar.io`) → « Afficher l'original » → rechercher une ligne d'en-tête supplémentaire (`Bcc:`, `Content-Type:`, `Subject:` replié…) ou un corps contenant une ligne `From:` / `Reply-To:` / `Content-Type: text/html`.
Le message est-il affiché dans un **interface web** de CyberAar (back-office, CRM, AarMe) ?	Si oui : vérifier si le contenu est rendu en HTML → l'injection devient alors un **XSS stocké dans l'interface de triage** (actif *Plateforme AarMe*, classe Critique) et non plus une simple déformation d'e-mail.

Aucune donnée réelle n'a été modifiée, aucun envoi massif n'a été réalisé : la campagne a respecté la limite de débit de la plateforme (une requête par test, espacée) et les règles « pas d'attaque par déni de service ».

4. Impact

Dimension	Évaluation
Canal de confiance détourné	Le message arrive depuis l'adresse officielle du site et l'interface d'administration affiche le nom et le contenu fournis par l'attaquant.
Forge d'identité dans le corps	Un bloc « De : … », une fausse signature, un faux avis « votre compte a été compromis » ou de faux liens sont injectables en fin de ligne de message.
En-têtes (à confirmer)	Si un champ atterrit dans un en-tête, l'attaquant peut ajouter un destinataire (capture de la notification) ou forcer un format HTML.

Robustesse	L'accusé de réception est malformé (nom tronqué) : l'utilisateur voit un message incohérent, source d'abandon et de support.
Ce qui n'est **pas** impacté	Aucune donnée de la plateforme, aucun compte, aucun rapport, aucune prime : l'impact reste dans la messagerie de l'équipe (classe **Standard**).

Score final CyberAar : sévérité technique CVSS v4.0 `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/S:X/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N` = **5,3** ; impact métier pondéré (portée, continuité de la décision de triage, absence d'impact sur les données de la plateforme) = **4/10** → score final √(5,3 × 4) = **4,6** → sévérité **Moyenne** (classe Standard).

Si l'e-mail est livré sans aucune interaction du destinataire, la sévérité technique passe à `UI:N` = **6,9** → score final √(6,9 × 4) = **5,3**. Si le message est affiché dans une interface web de CyberAar et que le contenu y est rendu en HTML, l'impact d'intégrité passe à `VI:H` = **8,8** → score final √(8,8 × 4) = **5,9**. Ces deux vérifications sont demandées au §3.3 et relèvent de l'équipe CyberAar.

5. Captures

(capture d'écran de l'accusé de réception à insérer : en-tête « De : CyberAar », corps « Bonjour, / Nous avons bien reçu votre message. » — le nom tronqué demonstrates l'injection.)

Pièces jointes à fournir avec le rapport : `1-reproduction-contact.sh`, `2-acuse-recu-original.eml` (afficher l'original), `3-acuse-recu-capture.png`.

6. Correction recommandée

Nettoyer systematically les valeurs fournies par l'utilisateur avant toute construction de message (une seule fonction suffit) :

```
const clean = (v) => String(v ?? '').replace(/[\r\n]+/g, ' ').trim();
```

À appliquer sur **tous** les champs de tous les formulaires et à effectuer avant toute utilisation, y compris dans les en-têtes. Un contrôle de type « nom de personne » (lettres, espaces, tirets) sur les champs qui ne doivent contenir que du texte libre est un complément utile mais ne remplace pas le nettoyage.

7. Annexe — surface explorée et contrôles vérifiés conformes

La campagne a covered l'ensemble de la surface publique et de la surface authentifiée du programme. Les éléments suivants ont été **vérifiés et sont conformes** : ils sont listés ici pour éviter que le même travail soit refait par un autre chercheur.

Contrôle	**Résultat mesuré**
CSRF sur les POST publics	Un `Origin`/`Refer` same-origin est exigé (sans → `403 forbidden`).
Adresses jetables	Refusées (`disposable_email`).
Enumeration de comptes au login	Message unique `invalid_credentials` (compte existant ou non) ; l'ordre de vérification est « mot de passe d'abord » → pas d'oracle ; pas d'oracle de timing non plus (257-304 ms vs 283-284 ms).
Réinitialisation de mot de passe	Réponse unique `{"success":true}` quelle que soit l'adresse (existante ou non) → pas d'énumération d'adresses.

Confirmation d'adresse	`{"error":"invalid_token"}` uniforme, y compris avec le mot de passe correct : la preuve du propriétaire est exigée.
Double authentification	Le code n'est accepté que **lié à la session** créée par l'étape précédente (un `totp` envoyé seul est ignoré) ; cookie de session `Secure; HttpOnly; SameSite=Lax`.
Codes de secours	Exigent le mot de passe courant **et** un code TOTP vivant (avec le vrai mot de passe + un faux TOTP → `invalid_code`, aucun code divulgué).
Session/Récompenses/pièces jointes	`kyc_required` / `identity_required` : les primes et les pièces jointes de rapport exigent une identité validée.
Mass assignment	Les champs inconnus sont ignorés (`nothing_to_update`) sur le profil ; `invalid_input` sur les formulaires.
Validation des champs affichés publiquement	`invalid_name` / `invalid_country` : aucune balise HTML injectable dans le profil public.
Upload d'avatar	Type sniffé (`image/png,jpeg,webp`), contenu **ré-encodé** par le serveur (une charge HTML ajoutée est supprimée), nom d'origine remplacé par un aléatoire à extension `.png`, servi avec `nosniff` et `CSP: default-src 'none'; sandbox` → XSS impossible.
SSRF	Le champ `profile` (seule URL acceptée) n'est jamais récupéré par le serveur, ni à l'inscription ni au rendu du profil (0 requête sortante observée).
XSS stocké public	CSP stricte sans `'unsafe-inline'` en `script-src` → les gestionnaires d'événements en attribut ne s'exécutent pas ; l'API rejette par ailleurs le contenu HTML dans les champs publics.
En-têtes de sécurité	CSP stricte, HSTS avec `preload`, `X-Frame-Options: DENY`, `nosniff`, `frame-ancestors 'none'`, `object-src 'none'`, `base-uri 'self'`.
Surface API	11 endpoints sur l'hôte AarMe (dont 3 publics), 2 sur l'API formulaires, 30 endpoints de compte (verbes correctes : `PATCH` profil, `DELETE` en-tête/avatar) → surface étroite, chaque endpoint de données sensibles protégé par session et/ou identité validée.
API publique du profil	N'expose que `avatar` (booléen), `country` et `game` — ni email, ni KYC, ni sanctions, ni URL de profil ; pseudo inexistant → `404` uniforme.

8. Pistes écartées (et pourquoi)

Piste	**Motif d'exclusion**
SSRF via l'URL de profil	Aucune récupération d'URL par le serveur (mesuré, 0 requête sortante).
XSS stocké dans le profil public	Validation serveur du nom et du pays + CSP sans `'unsafe-inline'`.
XSS via l'avatar (polyglotte image/HTML)	Le contenu est ré-encodé, renommé, servi en `image/png` avec `nosniff` et une CSP `sandbox`.
Énumération de comptes	Messages et latences uniformes sur le login et le reset.

Fuite de données par l'API publique	Aucun champ sensible exposé.
Contournement de quota / abus de masse	Le plafond de la plateforme est respecté : une requête par test, espacée de 12 minutes ; aucun envoi massif n'a été réalisé (une contrainte explicite des règles).

9. Limites de la démonstration

- La démonstration porte sur l'**insertion brute** de contenu dans l'e-mail (prouvée par l'accusé de réception tronqué). La propagation aux **en-têtes** et l'éventuel affichage dans une interface web relèvent d'une vérification interne : les étapes sont fournies au §3.3.
- Aucun impact n'a été cherché sur des données réelles, aucun compte tiers n'a été visé, aucun envoi en masse n'a été réalisé.
- Le compte de test utilisé est un compte de recherche créé par le chercheur, sur une adresse lui appartenant, avec son accord.